Diagramas UML GoVet 2

Nikolas Briceño Ramirez Alvaro Burgos Andrade Nicolás Sandoval Jerez

El presente informe busca comparar el diagrama de despliegue y el diagrama de secuencia para el caso de uso:

I-Respecto a su proyecto, revisar:

1. ¿Qué diagramas UML realizó? (Si no los tiene, crear 2 diagramas: uno de secuencia y otro de componentes o despliegue)

   En cuanto al proyecto de GoVet se realizaron los siguientes diagramas:

   - Diagrama de componentes.
   - Diagrama de Actividad
   - Diagrama de secuencia (una para cada historia de usuario).

2. ¿Cuáles de los diagramas le sirven para comunicar la arquitectura?

   El diagrama de componentes es el que sirve para comunicar la arquitectura.

II- Respecto al proyecto de su compañero(el que instaló):

1. Crear 2 diagramas (uno de secuencia y otro de componentes o despliegue)

1.1. Diagrama de secuencia para caso de uso “Buscar pacientes por su nombre, raza o dueño”.

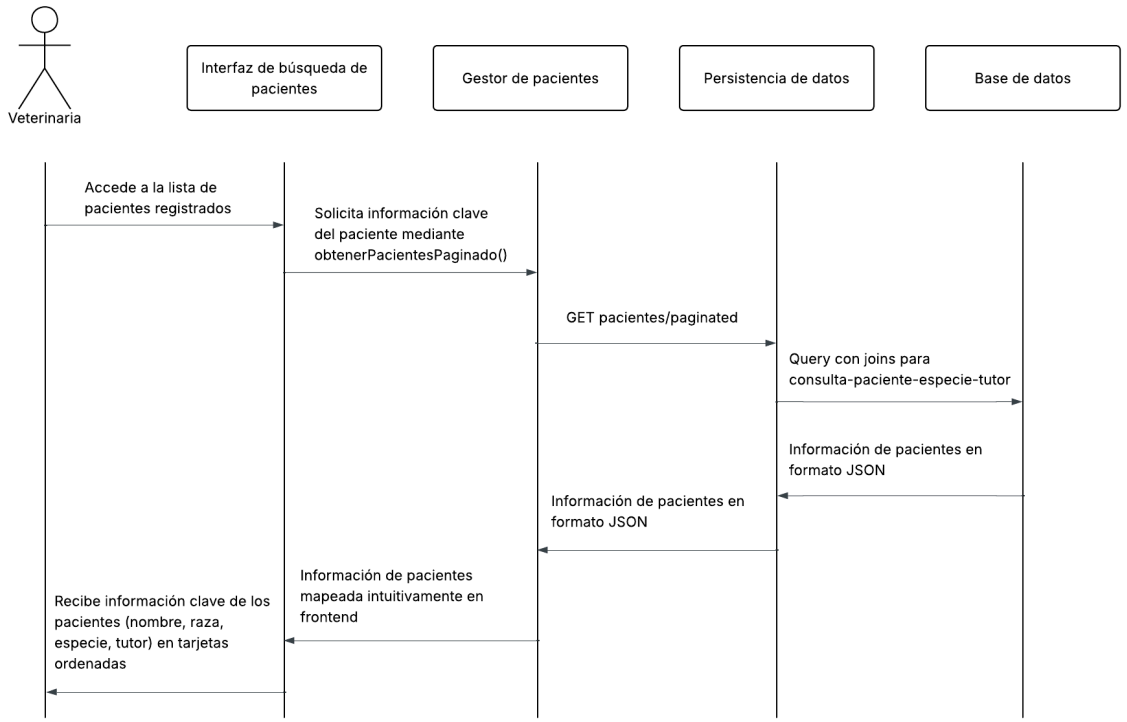


Figura 1. Diagrama de secuencia hecho por el grupo

En retrospectiva, el diagrama de secuencia debió haber sido construido con la estructura que el código proporciona, en lugar del diagrama de componentes.

1.2. Diagrama de despliegue

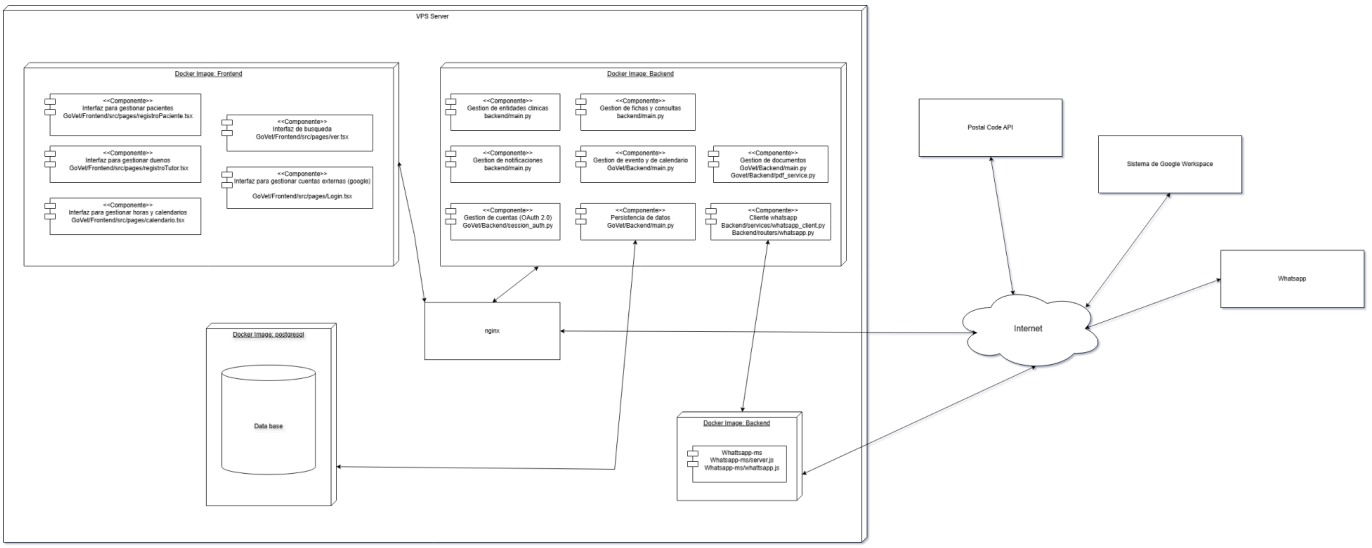


Figura 2. Diagrama de despliegue

3. El grupo original versiona en el repositorio sus diagramas. Luego cada grupo crea los mismos tipos de diagramas para el otro grupo asignado revisando el código del

sistema (sin revisar los diagramas originales). El grupo secundario contrasta los creados versus los originales ¿Que diferencias y similitudes existen?

3.1. Diagrama de secuencia

El diagrama de secuencia original del caso de uso “Buscar pacientes por nombre, raza o dueño” cuenta con mayor detalle en la interacción entre componentes. El hecho por este grupo tiene un nivel de abstracción mayor, rescatando las interacciones principales del software.

Link del diagrama:
https://trello.com/c/2uF3H7ah/14-hu-07-como-veterinaria-quiero-buscar-pacientes-tutores-y-fichas-con-su-información-clave-para-recuperar-información-importante-d

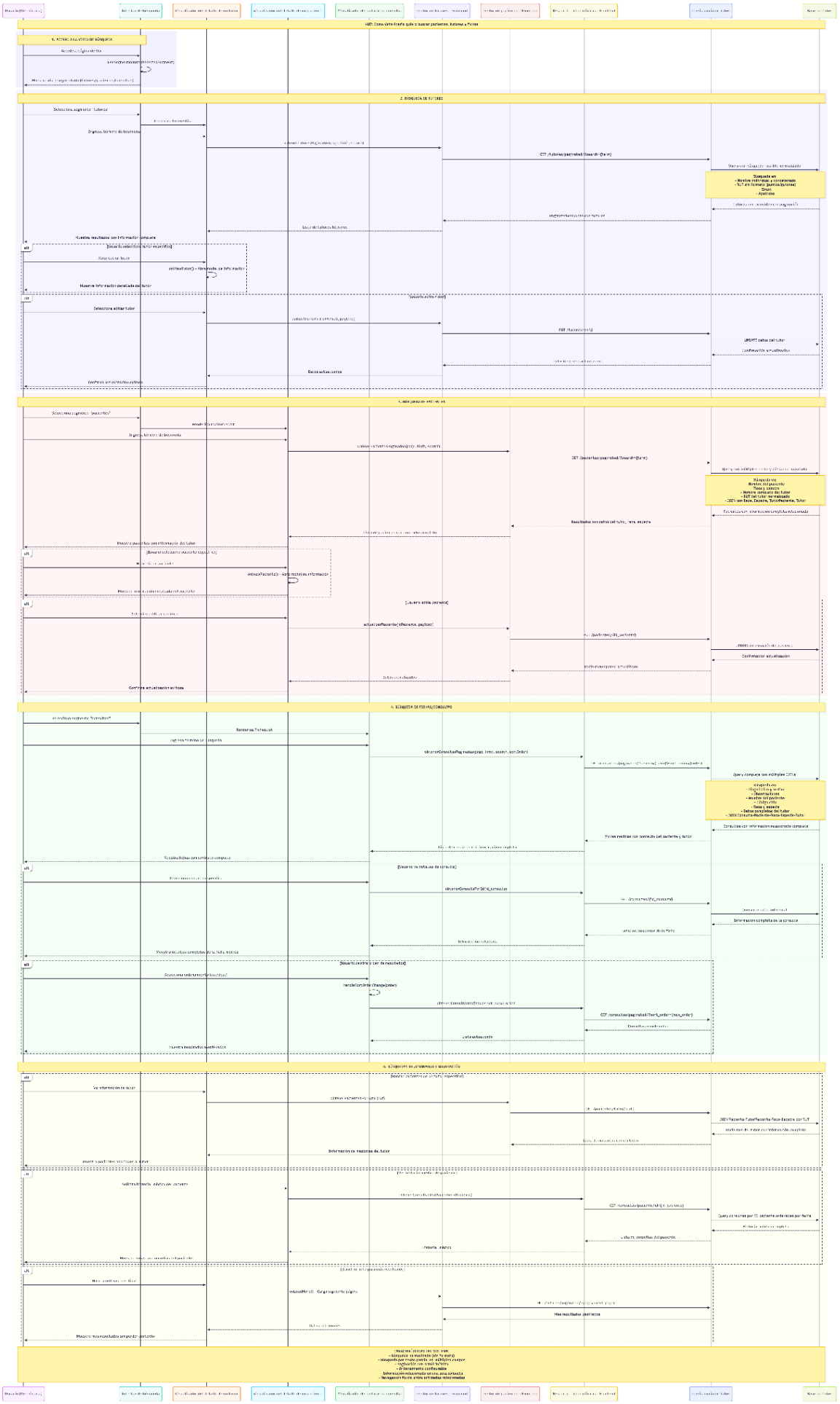
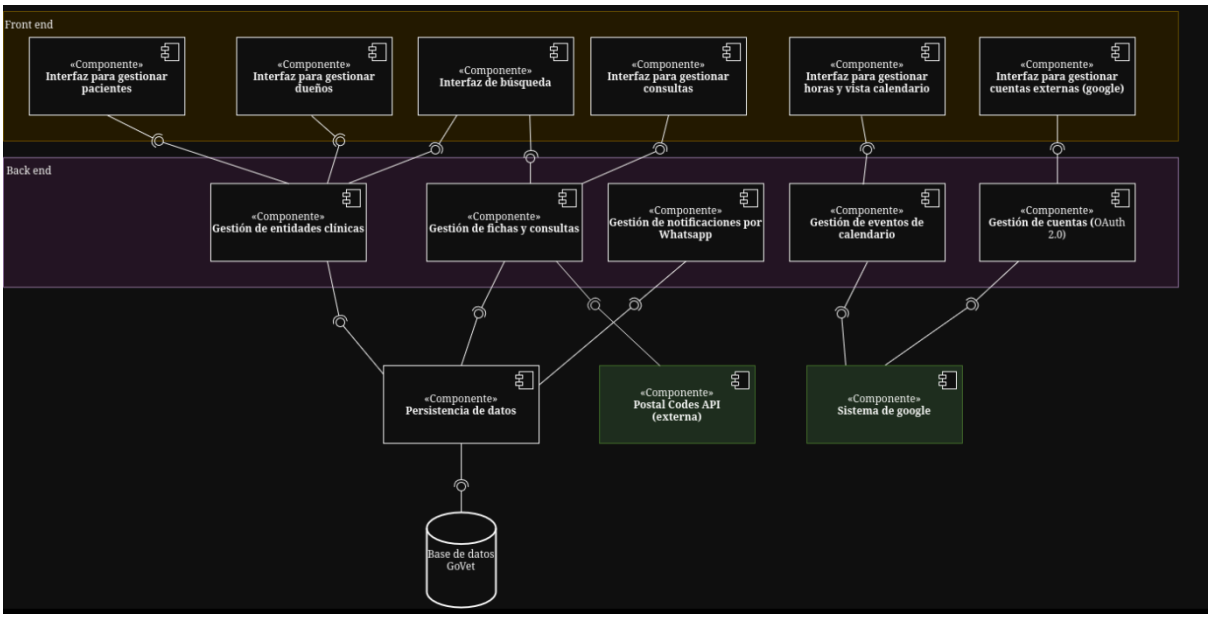

HUT: Como Vete Enseñando a buscar parientes, tutores y Pares
1. INICIO: Vista de Búsqueda
Accede a /Tutores
Inicializa componentes (Header, Footer, Sidebar)
Muestra vista inicial (Buscador y Lista de tutores)
2. BÚSQUEDA DE TUTORES
Selecciona asignatura / Tutoría
Envía formulario de búsqueda
Digitaliza formato de búsqueda
Solicitud de tutores (page=1, limit=12, search=)
GET /tutores/paginated/?search=(Q or nil)
Serializa resultados con relaciones
Diagnóstico:
- Muestra individual y correlacionado
- %T pre formato (puntuación/zona)
Tutores
Apellido
Calcula tutores disponibles (paginado)
registrados/tutores/reservados
Envía datos de tutores
Muestra resultados con info básica compacta
[Usuario selecciona tutor específico]
Click ver detalle
setShowTutor() - Abre modal con info completa
Muestra info completa del tutor
[Usuario edita perfil]
Selecciona editar tutor
consultarTutorPorId(id_tutor, pagina)
PUT /tutores/<id_tutor>
UPDATE datos del tutor
Confirma actualización
actualizado_correo
Envía datos actualizados
Confirma actualización exitosa
4. MODIFICACIÓN PARENTESCO
Selecciona asignatura / Tutoría
modificarParentesco
Digitaliza formato de búsqueda
Solicitud de parientes (page=1, limit=6, search=)
GET /parentesco/paginated/?search=(Q or nil)
Query con múltiples JOINs y filtros opcionales
Diagnóstico:
Muestra del paciente:
Tutor y correo
- Muestra compacta del tutor
- %T del tutor normalizado
- JOIN con Tutor, Parentesco, TutorParentesco, Tutor
Transforma a formato compatible con frontend
Resultados con datos del tutor, tipo_parentesco
Envía datos de parientes con relaciones
Muestra parientes con información del tutor
[Usuario selecciona parentesco específico]
click Modificar
editarParentesco() - Abre modal de edición
Muestra modal con datos del parentesco
[Usuario edita parentesco]
Submit cambios
actualizarParentesco(id_parentesco, payload)
PUT /parentesco/<id_parentesco>
UPDATE parentesco SET ...
Confirmación actualización
actualizado_correo
Envía datos actualizados
Confirma actualización exitosa
5. BÚSQUEDA DE PARIENTES/CONSULTAS
Accede a /consultas
Renderiza Footer.js
Inicializa componentes
obtenerConsultasPag(page=1, limit=, search=, sortOrder)
GET /consultas/paginated/?page=1&limit=&search=&sort_order=
Query compleja con múltiples JOINs
Diagnóstico:
- Diagnóstico
- Diagnóstico
- Observaciones
- Nombre del paciente
- Cédula
- Tutor
- Correo
- Apellido
- Datos del tutor
- JOIN Consulta, Paciente, Tutor, Tutor
Consultas con información relacionada compacta
Pares medicos con contacto del paciente y tutor
Envía consultas con relaciones completas
Muestra lista con tutores compactos
[Usuario ve detalles de consulta]
Click ver detalles
obtenerConsultaPorId(id_consulta)
GET /consultas/<id_consulta>
Consulta con relaciones completas
Información completa de la consulta
Información completa
Muestra detalles completos de la consulta
[Usuario cambia ordenamiento]
Selecciona ordenamiento
handleSortChange()
obtenerConsultasPag(1, limit, search, sortOrder)
GET /consultas/paginated/?sort_order=(new_order)
Consultas ordenadas
Datos actualizados
Muestra resultados ordenados
6. Diagnóstico en parientes y seguimiento
[Usuario ve parientes con tutor disponible]
Ver información de tutor
obtenerParentescosPorTutor()
GET /parentesco/tutor/<id_tutor>
JOIN Parentesco, TutorParentesco, Paciente, Consulta por %T
Información de parientes del tutor
Información de parientes del tutor
Muestra información del tutor
[Usuario ve historial médico del paciente]
Click ver historial médico
obtenerHistorialMedico(id_paciente)
GET /historial_medico/paciente/<id_paciente>
Query de recetas por ID paciente ordenadas por fecha
Recetas médicas ordenadas
Consultas del paciente
Historial médico
Muestra historial médico del paciente
[Usuario ve Diagnósticos Recientes]
Hace modificar con filtros
loadLastMonth() - Cargar diagnósticos página
GET /diagnosticos/?page=1&limit=&search=&sort
Más resultados disponibles
Diagnósticos
Muestra más resultados en grid/lista
- Búsqueda en múltiples JOIN's %ilike
- Búsqueda con multiples JOIN's %ilike
- Navegación por rutas parentesco, diagnóstico, cuerpo
- Reflejo de cambios en tiempo real
- Ordenamiento configurable
Información relacionada entre una consulta
Navegación fluida entre entidades relacionadas

Figura 3. Diagrama de secuencia original

3.2 . Diagrama de componentes/despliegue



- Whatsapp:
    - El diagrama de componentes no muestra ningún componente externo que maneje las notificaciones por whatsapp
    - El diagrama de despliegue realizado muestra el despliegue del componente de whatsapp-ms y su comunicación con whatsapp-client desplegado en el backend.
- Consistencia:
    - El diagrama de componentes no muestra el/los archivo/s que se encarga de la lógica para cada componente. Muestra una abstracción del sistema.
    - Diagrama de despliegue muestra el lugar de los archivos importantes desplegados.

El grupo destino no contaba con diagrama de despliegue. Por tanto estas diferencias son las más notables notadas entre ambos diagramas.